Python object model fundamentals Shared by surya

explain clearly this python concepts 3. Python Object Model
Everything is Object
Type Hierarchy
Metatype

Python Object Model

1. Everything is an Object

In Python, literally *everything* you can name or assign to a variable is an object — numbers, strings, functions, classes, modules, even types themselves. Every object has three things:

- **Identity** — a unique id (`id(obj)`, basically its memory address)
- **Type** — what kind of object it is (`type(obj)`)
- **Value** — the data it holds

python

```python
x = 5
print(type(x))      # <class 'int'>
print(id(x))         # some unique integer

def foo(): pass
print(type(foo))     # <class 'function'>

class Bar: pass
print(type(Bar))      # <class 'type'>   <-- even classes are objects!

print(type(int))      # <class 'type'>
print(type(type))     # <class 'type'>   <-- type is an object too
```

This means functions can be stored in variables, passed as arguments, put in lists, etc. — they're not special syntax, just objects like any other.

python

```
funcs = [print, len, str]   # functions stored in a list, like any objects
```

2. Type Hierarchy

Every object has a **type** (its class), and every type itself has a type. This forms a hierarchy:

```
instance  →  class  →  metaclass  →  type
   5      →   int   →    type     →  type
```

- `5` is an **instance** of `int`
- `int` is a **class**, and is itself an instance of `type`
- `type` is the **metaclass** — and it is an instance of itself

python

```
print(type(5))        # <class 'int'>
print(type(int))       # <class 'type'>
print(type(type))       # <class 'type'>  -> type is its own type, the chain sto
```

Separately, classes also have an **inheritance hierarchy** (`__bases__` / `__mro__`), which is different from the type chain above:

python

```
class Animal: pass
class Dog(Animal): pass

print(Dog.__mro__)
# (<class 'Dog'>, <class 'Animal'>, <class 'object'>)
```

`object` sits at the top of *every* inheritance chain — all classes ultimately inherit from `object`.

So there are two different relationships to keep straight:

- **"is-a" / inheritance** — `Dog` inherits from `Animal` (subclass relationship)
- **"is-an-instance-of" / type** — `Dog` is an instance of `type` (the type/instance relationship)

3. Metatype (Metaclass)

A **metaclass** is "the type of a type" — it's the class used to create classes, just as a class is used to create instances.

- Normal class → creates **instances**
- Metaclass → creates **classes**

```python
class MyClass:
    pass

obj = MyClass()        # MyClass creates an instance
print(type(obj))        # <class 'MyClass'>
print(type(MyClass))    # <class 'type'>  -> type created MyClass
```

`type` is the default metaclass for almost every class in Python. When you write `class Foo: ...`, Python actually calls `type(name, bases, namespace)` behind the scenes to build that class object.

You can prove this — these two are equivalent:

```python
class Foo:
    x = 1

# is the same as:
Foo = type('Foo', (), {'x': 1})
```

Custom metaclasses let you control how classes themselves are constructed — useful for things like enforcing rules on class definitions, auto-registering classes, or adding methods automatically.

```python
class Meta(type):
    def __new__(mcs, name, bases, namespace):
        print(f"Creating class {name}")
        return super().__new__(mcs, name, bases, namespace)

class MyClass(metaclass=Meta):
    pass
# Output: Creating class MyClass
```

Putting it together

Level	Example	Created by
Instance	`5`, `"hi"`, `obj`	its class
Class	`int`, `str`, `MyClass`	its metaclass (usually `type`)
Metaclass	`type`	itself

This is why Python is often called a **fully object-oriented, dynamic language with a uniform object model**: classes aren't a special compile-time construct — they're regular runtime objects, created by other objects (metaclasses), all the way up, with `type` sitting at the top as the metaclass that creates itself.

Object Lifecycle
Attribute Lookup
Method Resolution
Descriptor Protocol

Python Object Model — Part 2

1. Object Lifecycle

Every object goes through three phases: **creation**, **use**, and **destruction**.

Creation: `__new__` then `__init__`

- `__new__(cls, ...)` — actually **creates** the raw object (allocates memory, returns the instance). This is a `staticmethod`.
- `__init__(self, ...)` — **initializes** the already-created object (sets attributes). Returns `None`.

python

```python
class Point:
    def __new__(cls, *args, **kwargs):
        print("1. __new__ called - creating the object")
        instance = super().__new__(cls)
        return instance

    def __init__(self, x, y):
        print("2. __init__ called - initializing the object")
```

```
        self.x = x
        self.y = y

p = Point(3, 4)
# 1. __new__ called - creating the object
# 2. __init__ called - initializing the object
```

`__new__` is rarely overridden in normal code — it matters for immutable types (`int`, `str`, `tuple`) or metaclasses, where the object must be fully formed *before* `__init__` runs.

Destruction: `__del__` and garbage collection

Python uses **reference counting** — when an object's reference count hits zero, it's destroyed. `__del__` is the destructor hook, called right before memory is reclaimed.

python

```
class Resource:
    def __del__(self):
        print("Resource being destroyed")

r = Resource()
del r          # triggers __del__ (reference count -> 0)
# Resource being destroyed
```

Python also has a **cyclic garbage collector** to clean up reference cycles (e.g., two objects pointing at each other) that reference counting alone can't catch.

python

```
import sys
x = []
print(sys.getrefcount(x))   # check ref count (will be 2: x + the argument)
```

Lifecycle summary:

```
__new__()  →  __init__()  →  ... object lives, used by program ...  →
refcount=0  →  __del__()  →  memory freed
```

2. Attribute Lookup

When you write `obj.name`, Python follows a defined search order. This is governed by `__getattribute__`, which is called for *every* attribute access.

The lookup order is:

1. **Data descriptors** found on the type (explained below) — checked first
2. **Instance** `__dict__` — the object's own attributes
3. **Non-data descriptors and class attributes** — found on the type (and its MRO)
4. `__getattr__` — only called as a *fallback* if all the above fail

python

```python
class Demo:
    class_attr = "I'm on the class"

    def __init__(self):
        self.instance_attr = "I'm on the instance"

d = Demo()
print(d.instance_attr)   # found in instance __dict__
print(d.class_attr)      # not in instance __dict__, found on the class

print(d.__dict__)         # {'instance_attr': "I'm on the instance"}
print(Demo.__dict__.keys())  # contains 'class_attr', '__init__', etc.
```

`__getattr__` only fires when normal lookup fails completely:

python

```python
class Demo:
    def __getattr__(self, name):
        return f"{name} not found, here's a default"

d = Demo()
print(d.whatever)   # "whatever not found, here's a default"
```

Contrast with `__getattribute__`, which intercepts *every* attribute access (even ones that exist) — powerful but easy to create infinite recursion with if you're not careful.

3. Method Resolution (MRO)

When a class inherits from multiple classes, Python needs a deterministic order to search for methods/attributes across the inheritance chain. This order is the **Method Resolution Order (MRO)**, computed using the **C3 linearization algorithm**.

python

```python
class A:
    def greet(self):
        return "Hello from A"
```

```python
class B(A):
    def greet(self):
        return "Hello from B"

class C(A):
    def greet(self):
        return "Hello from C"

class D(B, C):
    pass

d = D()
print(d.greet())          # "Hello from B"
print(D.__mro__)
# (D, B, C, A, object)
```

Python searches `D → B → C → A → object`, left to right, depth-first but flattened to avoid visiting `A` twice (this is the classic **diamond problem**, which C3 linearization solves cleanly).

`super()` uses the MRO too — it doesn't mean "my parent class," it means "the next class in the MRO":

python

```python
class B(A):
    def greet(self):
        return super().greet() + " + B"   # goes to whatever's next in MRO, not
```

You can inspect the MRO anytime with `ClassName.__mro__` or `ClassName.mro()`.

4. Descriptor Protocol

A **descriptor** is an object that customizes attribute access by defining any of:

- `__get__(self, instance, owner)` — called on attribute read
- `__set__(self, instance, value)` — called on attribute write
- `__delete__(self, instance)` — called on attribute deletion

Descriptors live on the **class**, not the instance, and they're how `property`, methods, `staticmethod`, and `classmethod` all work under the hood.

python

```python
class PositiveNumber:
    def __set_name__(self, owner, name):
        self.name = "_" + name

    def __get__(self, instance, owner):
        return getattr(instance, self.name, None)

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError("Must be positive")
        setattr(instance, self.name, value)

class Account:
    balance = PositiveNumber()   # descriptor instance, lives on the class

    def __init__(self, balance):
        self.balance = balance   # triggers __set__

a = Account(100)
print(a.balance)     # triggers __get__ -> 100
a.balance = -5        # triggers __set__ -> raises ValueError
```

Data vs Non-data descriptors

- **Data descriptor:** defines `__set__` and/or `__delete__` (with or without `__get__`) — takes priority over instance `__dict__`.
- **Non-data descriptor:** defines only `__get__` — instance `__dict__` takes priority over it.

This is *exactly* why functions become bound methods automatically:

python

```python
class Foo:
    def bar(self):
        return "hi"

f = Foo()
print(f.bar)   # <bound method Foo.bar of <Foo object>>
```

Functions implement `__get__`, so when accessed through an instance, they return a "bound method" object that automatically passes `self`. That's the entire mechanism behind Python's method calls — no special-casing, just the descriptor protocol.

`property` is just a built-in descriptor:

```python
class Foo:
    @property
    def value(self):
        return 42

# roughly equivalent to:
class Foo:
    value = property(lambda self: 42)
```

How it all connects

```
obj.attr access
   → __getattribute__ runs
       → checks type's MRO for a data descriptor → calls __get__ if found
       → else checks instance __dict__
       → else checks type's MRO for class attr / non-data descriptor → calls
__get__ if found
       → else falls back to __getattr__
```

This is the real machinery behind `property`, `@staticmethod`, `@classmethod`, ORMs (like Django models), and validated attributes — all built on the same descriptor protocol.